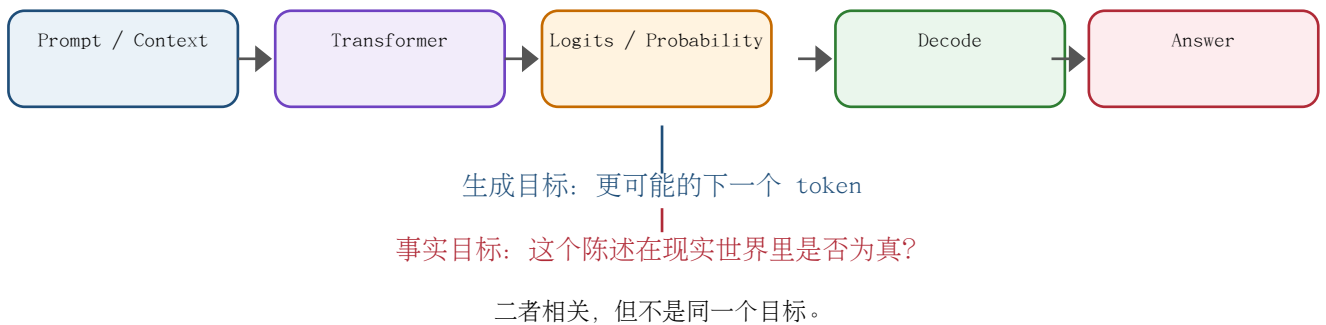


第24讲: Hallucination

为什么大语言模型可以说得非常像真的，却仍然是错的？

本讲目标

学完后你应该能从底层解释：为什么“语言上顺滑”与“事实为真”不是一回事；能区分主要幻觉来源；知道 RAG、工具调用、验证器、校准与拒答各自解决什么问题；并能把幻觉问题转化为可测量的 Agent Systems 可靠性问题。



先记住一句: LLM 的基本训练目标是让下一个 token

更可能，而不是直接让每一句话都通过现实世界的事实审计。两者高度相关，但并不等价。

本讲结构: ①什么叫幻觉; ②底层原因; ③主要类型; ④为什么模型会“自信地错”; ⑤如何测量; ⑥RAG / Tool / Verifier 等缓解方法; ⑦与 Agent Systems 的关系; ⑧实验、练习与答案。

1. 什么叫 Hallucination?

在大语言模型语境里，Hallucination（幻觉）通常指：模型生成了看起来合理、语言流畅，但缺乏依据、与给定证据冲突，或者事实上错误的内容。不同论文对边界定义略有差异，因此做科研时必须先写清自己的操作性定义。

通俗理解		
模型不是“先访问真相数据库，再把真相翻译成句子”。它是在当前上下文条件下计算下一个 token 的概率，并不断继续生成。只要一条错误路径在语言分布上很顺，它就可能被完整地续写出来。		
现象	例子	为什么危险
捏造事实	编出不存在的人名、机构、日期或数值	用户很难从流畅语言中察觉
捏造引用	给出不存在的论文题目、DOI、页码	科研场景尤其高风险
证据冲突	材料明确写 A，回答却说 B	说明模型没有被证据充分约束
无依据补全	信息不足时仍给出具体数字	把不确定性伪装成确定性
常见误区		
“幻觉”并不等于模型出现了人类意义上的知觉体验。这里是工程/研究术语，描述的是输出与事实或证据之间的失配。		

2. 最关键的底层原因：训练目标不是“真值判定器”

对一个自回归语言模型，可以把最基础的训练目标写成：

$$P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

也就是：给定前面的 token，预测下一个 token。训练时，如果正确 token 的概率太低，就产生更大的 Cross-Entropy Loss，再通过反向传播修改参数。

核心区别
“这个 token 在语料分布里是否很可能出现”与“这个命题在现实世界里是否为真”不是同一个问题。模型可以学到大量事实模式，但它的基础目标函数并没有在每次生成时调用一个外部真值函数。

例如，假设模型看到很多格式为“某某大学成立于____年”的文本。即使它不知道某个冷门大学的真实建校年份，它也可能根据语言模式补出一个看起来很合理的年份。

因此，幻觉不能简单解释成“模型太笨”或“模型训练不够”。更准确地说：生成系统的优化目标、内部知识状态与现实世界真值之间存在结构性间隙。

3. 为什么“语言正确”会伪装成“事实正确”

人类阅读时很容易把三种属性混在一起：

属性	含义	是否等价
Fluency	语言流畅、语法自然	不等于真实
Plausibility	听起来符合常识或模式	不等于真实

属性	含义	是否等价
Truthfulness	陈述与事实或可靠证据一致	需要外部检查

LLM 在 Fluency 和 Plausibility 上往往非常强，因此一个错误答案可能同时具备：专业术语、完整因果链、正确格式和肯定语气。

科研提醒
“写得像论文”与“有可核验来源”必须分开。科研场景中，任何关键文献、数据、公式来源、实验结果都应回到原论文、数据库或真实实验记录核验。

4. 幻觉从哪里来？四层来源模型

为了便于分析，可以把幻觉来源分成四层。这不是唯一分类，但非常适合做实验设计。

层	典型原因	例子
数据层	数据错误、冲突、稀疏、过时	同一事实存在多种互相矛盾的网页
模型层	容量有限、压缩与泛化误差	冷门事实没有被可靠编码
上下文层	信息缺失、冲突、过长、检索失败	正确证据没被放进当前 context
解码/系统层	采样、工具失败、链式错误	一次错误 tool result 被后续多轮继续放大

重要
不要把所有错误都叫“模型幻觉”。例如工具 API 返回了错误结果、数据库过期、RAG 检索错文档、Agent 把旧 Memory 当成新事实，这些都是系统级错误来源。

5. 三类特别常见的幻觉

- 5.1 参数知识型错误：模型只依赖参数里学到的知识回答，但知识缺失、模糊或冲突。
- 5.2 Grounding 失败：明明提供了材料，模型却没有忠实使用，或者把材料外的内容混进来。
- 5.3 Agent / Tool 链式幻觉：模型先做出错误假设，随后规划、工具选择、检索、总结都建立在错误前提上，导致错误被逐步放大。

例子
一个 Agent 误以为某文件已经存在，于是后续连续执行“读取文件→分析内容→生成结论”。如果 Runtime 没有真正检查文件是否存在，系统可能产生一整条看似连贯的虚假 workflows。

6. 为什么模型会“自信地错”？

首先要区分两个东西：token probability 与 事实置信度。模型对下一个 token 给出高概率，只说明在当前内部状态和上下文下，这个 token 很符合模型分布；它并不自动等价于“整个命题有 95% 的概率为真”。

例如一个句子已经生成：

The paper was published in 20

此时模型可能对“24”这个后续 token 很有把握。但这只是局部序列概率，不是对“论文确实发表于 2024 年”做了独立事实审计。

校准 Calibration
一个系统如果在声称 80% 置信度的题目上长期只有约 80% 正确率，我们称它更接近“校准”。LLM 的自然语言自信语气不应该被直接当成经过校准的概率。

7. Sampling 会不会制造幻觉？

Temperature、Top-k、Top-p 会改变生成路径的随机性，因此会影响某些错误的发生概率。但把 Temperature 调低并不能从根本上消灭幻觉。

如果模型内部最高概率答案本来就是错的，那么 Greedy Decoding 会非常稳定地输出同一个错误。

设置	可能效果	不能保证
低 Temperature	更稳定、更集中	不保证事实正确
高 Temperature	更多样、更探索	不保证更聪明
Greedy	每步取最高概率 token	最高概率命题仍可能是错的

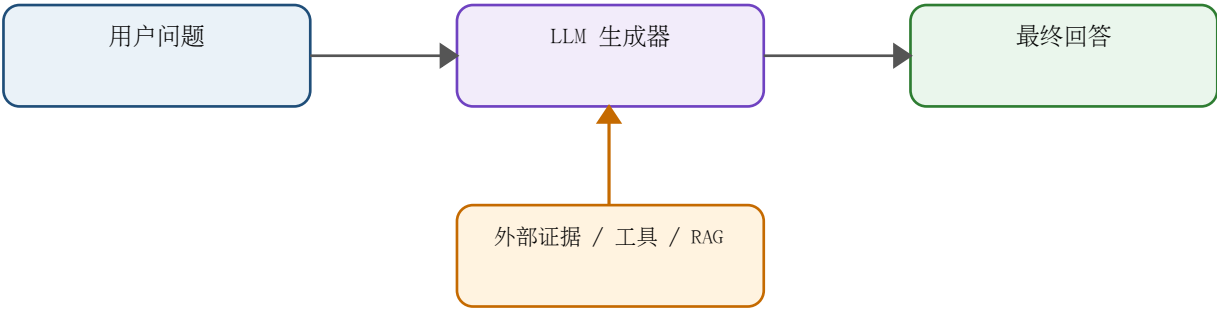
8. Context 不足与 Context 冲突

LLM 在当前请求中真正能使用的外部事实，首先必须进入当前 Context。若事实根本没进入上下文，模型只能依赖参数知识或猜测。

反过来，上下文越长也不意味着越可靠：过多无关内容可能形成 Context Bloat，多个冲突来源可能让模型选择错误版本，中间位置的重要证据可能被弱化。

与第21讲连接
Context Window 是工作台，不是长期记忆。把更多材料塞上工作台，并不自动等于模型会准确找到并正确使用关键证据。

9. RAG 为什么能降低一部分幻觉？



Grounding: 让生成被可检查证据约束

RAG (Retrieval-Augmented Generation) 的核心思想不是“让模型记住更多”，而是：先找证据，再把证据放进当前 Context，再要求模型基于证据回答。

Question -> Retrieval -> Evidence -> LLM -> Grounded Answer

但 RAG 不是万能药
如果检索到错误文档、文档过期、召回漏掉关键证据，或者模型忽略证据，RAG 仍然会失败。RAG 把问题从“参数里有没有事实”部分转化成“检索 + 证据使用是否可靠”。

10. Tool Use 为什么能减少“猜答案”

如果问题涉及实时天气、数据库记录、文件状态、计算结果、代码执行等可外部验证的事实，最可靠的路线往往不是要求模型“凭记忆回答”，而是调用工具。

问题类型	只靠参数知识	更可靠的系统做法
今天温度	容易过时	天气 API
账户余额	模型不应猜	授权数据库查询
精确算术	可能出错	计算器 / Python
某文件是否存在	不能凭语言推断	文件系统检查

Agent 原则
能验证的状态，尽量验证；能调用工具得到的事实，不要要求模型凭语言先验猜。

11. Verifier: 生成器和检查器分工

一个重要系统设计思想是把“生成答案”和“检查答案”分开:

Generator -> Candidate -> Verifier -> Accept / Reject / Revise

Verifier 可以是规则、程序、数据库、一致性检查器、另一个模型、单元测试、形式化证明器等。

这与第18讲 Test-Time Compute

相连: 额外计算真正有价值的地方, 往往不是让生成器重复说更多话, 而是增加独立的验证信号。

局限

Verifier 也会错。如果 Generator 和 Verifier 共享同样的盲点, 系统可能出现“错误共识”。因此研究时还要关注 error correlation。

12. Abstention: 不知道时应该允许“不知道”

很多系统的错误来自一个隐含假设: 模型必须永远给出完整答案。更可靠的系统应该允许:

-> ##### -> ##### / #### /

这叫做 Abstention (弃答/拒答) 思路。它通常需要权衡 Coverage 与 Accuracy: 弃答更多, 覆盖率下降, 但剩下答案可能更可靠。

策略	覆盖率	潜在准确率	适合场景
强制回答	高	可能较低	低风险聊天
低置信度弃答	中	更高	科研/专业问答
必须有证据才回答	更低	可显著提高	高风险事实任务

13. 如何测量 Hallucination?

科研中不要只说“感觉幻觉变少了”。必须定义可复现的测量。常见维度包括:

指标	测什么	示例
Factual Accuracy	事实是否正确	答案与金标准匹配
Faithfulness	是否忠于提供证据	回答是否被材料支持
Citation Precision	引用是否真的支持陈述	每条引用核验
Abstention Quality	不知道时能否正确弃答	选择性准确率
Calibration	置信度是否和真实正确率匹配	ECE / reliability curve
Error Correlation	多个模型/Agent 是否一起错	相关系数/重叠率

研究方法

先定义错误单元, 再定义评审协议。比如“一个答案包含 5 个原子事实, 其中 1 个无证据”, 和“整段答案判错”会得到完全不同的指标。

14. 一套实用的错误分解表

问题	检查点
模型是否知道？	参数知识中可能存在还是完全冷门？
证据是否进入 Context？	检索是否召回、是否被截断？
证据是否可靠？	来源是否过时、冲突、低质量？
模型是否遵循证据？	是否加入材料外内容？
工具是否正确执行？	API / 文件 / 数据库结果是否有效？
Verifier 是否独立？	是否共享同一盲点？
系统是否允许弃答？	证据不足时是否仍被迫给结论？

15. 为什么“让模型自己反思”并不总能解决幻觉

Self-Reflection、Self-Critique 有时能修正明显错误，但它们不是独立真值来源。如果第一次答案来自同一套错误内部表征，第二次“反思”可能只是把错误写得更完整。

关键原则

反思最有价值的情况通常是引入了新信息：工具执行结果、检索证据、单元测试、外部 verifier、另一种模型/数据源等。单纯增加相同模型的语言循环，独立性可能很弱。

16. Hallucination 与 Agent Systems：错误会如何被放大

普通单轮问答中的一个错误，可能只影响一句回答；Agent 系统中的一个错误，却可能进入状态、Memory、计划和工具链。

False Assumption -> Plan -> Tool Call -> Memory Write -> Next Decision -> ...

于是形成错误传播（error propagation）。如果错误被写入长期 Memory，它甚至可能在未来多个 episode 中持续存在。

科研连接

这正是 AI Systems Science 的典型问题：不要只问“某次回答是否 hallucinate”，还要问“错误是否被检测、是否被放大、是否写入状态、是否跨 Agent 传播、是否形成错误共识”。

17. 多 Agent：讨论能消灭幻觉吗？

不一定。多个 Agent 可以提供冗余、交叉检查和多样视角，但如果它们：

□□□ + □ Prompt + □□□□ + □ Judge

那么错误可能高度相关。此时多数投票只是把同一个错误放大。

条件	对集体纠错的影响
错误独立/低相关	投票、交叉验证更有价值
错误高度相关	多数票也可能稳定地错

条件	对集体纠错的影响
Agent 有不同工具/证据	可能形成互补信息
共享同一错误 Memory	可能出现错误同步传播
与你后续研究直接相关 真正值得测的不是“多 Agent 是否比单 Agent 高几分”，而是：Experience / Tool / Memory 的差异是否降低了 Error Correlation，并因此带来 Collective Gain。	

18. 一套工程上更可靠的回答流程

对于需要高事实性的任务，可以把系统设计成：

1) 用户输入 2) Retrieval / Tool 3) 模型生成 4) 模型验证 5) Verifier 6) 模型输出 7) 模型 Abstain

最低闭环
如果你以后做 Agent 可靠性实验，至少保留：输入、检索结果、工具结果、生成答案、验证结果、最终状态。没有这条审计链，很难判断错误究竟发生在哪一层。

19. 常见误区汇总

误区	更准确的理解
模型高概率输出 = 高事实置信	token 概率与命题真值概率不是一回事
Temperature=0 就没有幻觉	可以稳定地输出同一个错误
上下文越长越不幻觉	长上下文也会检索失败、冲突、稀释关键证据
RAG 一定正确	检索和证据使用都可能失败
多 Agent 投票一定更可靠	若错误高度相关，多数也会一起错
Self-Reflection 就是验证	没有新证据时独立性有限
所有错误都叫 hallucination	系统错误、工具错误、数据错误应分层定位

20. 小实验：语言先验 vs 外部证据

材料包里的 Python 实验构造了一个极小的“封闭世界事实表”。为了避免现实知识干扰，我们使用虚构实体。

模型有一套语言先验：某些名字更“像答案”，因此在没有检索时会偏向这些高概率名字；但真实事实表可能是另一个名字。

策略	机制
Prior-only	只按语言先验选答案
Retrieval-grounded	先查事实表，再回答
Noisy retrieval	检索有一定概率拿错证据

你会看到：Grounding 可以大幅降低错误，但检索质量本身又成为新的可靠性瓶颈。

21. 练习题

- 1) 为什么 “next-token prediction” 与 “事实真值判断” 不是同一个目标？
- 2) 为什么 Temperature=0 仍然可能稳定地产生错误答案？
- 3) Context Window 变大为什么不能直接等价于幻觉变少？
- 4) RAG 可以把哪类问题变得更容易验证？它又引入了什么新失败点？

- 5) 为什么“模型自己反思一次”不是独立 verifier?
- 6) 什么叫 Abstention? 它与 Coverage / Accuracy 有什么关系?
- 7) 多 Agent 投票在什么条件下更有效?
- 8) 如果一个 Agent 把错误事实写入长期 Memory, 后面可能发生什么?
- 9) 设计一个最小实验, 区分“多 Agent 协作效果”和“单纯更多 test-time compute”。
- 10) 为什么做 Agent 可靠性论文时应该报告 error correlation?

答案见材料包《03_第24讲_练习题与答案.pdf》。

22. 本讲总结

一句话总纲
LLM 的语言生成能力很强, 但“生成得像真的”与“经过证据验证后是真的”之间始终存在一道系统工程鸿沟。可靠系统的关键不是幻想这道鸿沟自动消失, 而是设计检索、工具、验证、校准、弃答和审计链去跨越它。

你应该带走的概念	最短解释
Hallucination	输出与事实/证据不一致或无依据
Grounding	用外部可核验证据约束生成
Verifier	独立检查候选答案
Calibration	置信与真实正确率的一致程度
Abstention	证据不足时允许不回答
Error Correlation	多个模型/Agent 是否倾向一起错
Error Propagation	错误在 Agent 状态、工具链、Memory 中继续扩散

下一讲: 第25讲《LLM 到底有没有“记忆”? ——参数记忆、Context、KV Cache、RAG 与 Agent Memory》。